

Barrer du texte et des équations dans Typst

— exploration des méthodes

Comment lire ce document. Chaque « Cas » montre un extrait de code Typst, puis son rendu réel, puis un verdict (✓ fonctionne / ✗ casse / △ fonctionne avec une réserve). Les sections sont numérotées (1.1, 1.2, ...) pour que vous puissiez me dire directement « le cas 3.4 ne va pas » ou « je choisis la méthode B pour les équations, A pour le texte ». Rien dans `src/` n'est modifié par ce document — c'est un espace d'essai, pas une implémentation.

1 Le constat de départ, précisé

L'intuition de départ était : « les équations **non inline** (`$ ... $` avec espaces) ne sont pas correctement barrées, contrairement aux équations inline (`$...$`) ». En testant méthodiquement, le constat réel est légèrement différent — la portée du bug est plus large qu'il n'y paraît :

`strike()` et `underline()` natifs de Typst ne décorent jamais les vrais glyphes mathématiques (variables, chiffres, opérateurs — β , X , $+$, $=$, exposants...), **que l'équation soit inline ou display**. Seul le texte entre guillemets à l'intérieur d'une équation (`x_ "senior"`, utilisé pour les sous-scripts nommés) est un vrai texte Typst et se fait donc décorer normalement. C'est pour ça qu'une équation avec plusieurs sous-scripts textuels **a l'air** partiellement barrée/soulignée dans le manuscrit réel (`examples/fridge-study`, `examples/emoji-email`) — mais les symboles mathématiques eux-mêmes ne le sont jamais, il suffit de zoomer pour le voir. Une équation display, plus grande et isolée sur sa propre ligne, rend cette absence beaucoup plus visible qu'une petite équation inline noyée dans une phrase déjà barrée — d'où l'impression initiale que seul le cas **display** posait problème.

Les quatre cas ci-dessous le montrent directement.

Cas 1.1 — équation inline seule, `strike()`

```
#strike($e = m c^2$)
```

$e = mc^2$

✗ Casse — aucune barre nulle part, alors que c'est une équation **inline**.

Cas 1.2 — équation inline dans une phrase, `strike()`

The relation `#strike($e = m c^2$)` holds for all observers.

The relation $e = mc^2$ holds for all observers.

✗ Casse — toujours rien sur l'équation, seul le reste de la phrase n'est de toute façon pas barré ici (il n'est pas dans le `strike()`).

Cas 1.3 — équation display seule, `strike()`

```
#strike($ e = m c^2 $)
```

$$e = mc^2$$

✗ Casse — même constat, sans surprise.

Cas 1.4 — texte + équation display + texte, tout enveloppé dans un seul `strike()` (reproduit `#rep/#del` du package)

```
#strike[was modeled as
$ P(x) = beta_0 + beta_1 X_ "senior" + beta_2 X_ "length" $
with $X$ the vector of covariates.]
```

~~was modeled as~~

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

~~with X the vector of covariates.~~

△ **Fonctionne, mais...** — le texte autour est bien barré ; sur l'équation, seuls les sous-scripts textuels (« senior », « length ») le sont — les symboles (P , x , β_0 , β_1 , X , $+$, $=$) ne le sont jamais. C'est très exactement ce qu'on voit aujourd'hui dans les manuscrits réels du package.

2 Méthode A — `strike()`/`underline()` natifs (comportement actuel du package)

C'est ce que `del-style/add-style` utilisent aujourd'hui (`src/marks.typ`). Fonctionne très bien sur du texte, quelle que soit sa longueur — c'est le comportement qu'on veut absolument garder pour la prose. Échoue silencieusement (aucune erreur, juste rien de visible) sur les vrais symboles mathématiques, `inline` ou `display`.

2.1 Texte

Cas A.1 — texte court

```
#strike[The treatment has an effect on survival.]
```

~~The treatment has an effect on survival.~~

✓ **Fonctionne** —

Cas A.2 — texte long, entièrement barré (reflow sur plusieurs lignes)

```
#strike[The treatment was associated with a clinically meaningful
reduction in the primary composite outcome, driven mainly by a
reduction in cardiovascular death rather than non-fatal myocardial
infarction, though the confidence interval remained fairly wide given
the modest sample size.]
```

~~The treatment was associated with a clinically meaningful reduction in the primary composite outcome, driven mainly by a reduction in cardiovascular death rather than non-fatal myocardial infarction, though the confidence interval remained fairly wide given the modest sample size.~~

✓ **Fonctionne** — le retour à la ligne se fait normalement, mot par mot, comme n'importe quel paragraphe justifié.

Cas A.3 — texte long, partiellement barré (début barré, fin normale)

```
#strike[This entire opening clause of the sentence is what we want
removed, since it repeats information already given earlier and adds
nothing new for the reader to consider here today,] but the remainder
of the sentence, which introduces the actual new finding, should stay
completely normal.
```

~~This entire opening clause of the sentence is what we want removed, since it repeats information already given earlier and adds nothing new for the reader to consider here today,~~ but the remainder of the sentence, which introduces the actual new finding, should stay completely normal.

✓ **Fonctionne** — la partie barrée et la partie normale se réenroulent chacune indépendamment, sans se gêner.

2.2 Équations

Cas A.4 — équation display courte

```
#strike($ E = m c^2 $)
```

~~$E = mc^2$~~

✗ **Casse** — rien de visible.

Cas A.5 — équation display longue / large

```
#strike($ P("y" | X) = "logit"^(-1)(beta_0 + beta_1 X_1 + beta_2 X_2 + beta_3 X_3) $)
```

~~$P(\mathfrak{y} \mid X) = \text{logit}^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$~~

✗ **Casse** — rien de visible non plus, quelle que soit la largeur.

Cas A.6 — équation display sur deux lignes

```
#strike($ a + b + c + d \ + e + f + g + h $)
```

~~$$\begin{aligned} &a + b + c + d \\ &+ e + f + g + h \end{aligned}$$~~

✗ **Casse** —

Cas A.7 — un seul terme barré à l'intérieur d'une équation

```
$ beta_0 + #strike($beta_1 X_1$) + beta_2 X_2 $
```

~~$\beta_0 + \beta_1 X_1 + \beta_2 X_2$~~

✗ **Casse** — le terme reste correctement formaté en italique mathématique — `strike(...)` n'empêche pas ça — mais toujours aucune barre.

3 Méthode B — superposition maison (« mark-line »), mesure + boîte + ligne

C’est l’approche déjà tentée et abandonnée en M1 (voir CLAUDE.md §6bis) : mesurer le contenu (measure()), l’envelopper dans une box de la taille mesurée, et superposer une vraie line() par-dessus via place()).

```
#let strike-b(body) = context {
  let sz = measure(body)
  box(width: sz.width, height: sz.height)[
    #body
    #place(top + left, dy: sz.height / 2, line(length: sz.width, stroke: 0.6pt))
  ]
}
```

3.1 Texte — c’est ici que ça casse (déjà documenté, revérifié)

Cas B.1 — texte court

The treatment has an effect on survival.

✓ **Fonctionne** — fonctionne, mais un texte court ne teste rien d’intéressant.

Cas B.2 — texte long, entièrement barré — LE bug déjà connu

This is a fairly long sentence that should wrap across more than one line when placed inside

✗ **Casse** — measure() sans contrainte de largeur mesure le texte comme s’il tenait sur **une seule ligne infiniment large** — la box résultante est aussi large que ce texte à plat, donc soit elle déborde de la page (ce qui se passe ici, coupé par le cadre rouge), soit, si on lui impose une largeur, le texte ne se réenroule plus du tout à l’intérieur (il est devenu un bloc atomique). C’est exactement la régression décrite dans CLAUDE.md §6bis qui a fait abandonner cette piste pour le texte.

Cas B.3 — texte long, partiellement barré

This entire opening clause of the sentence is what we want removed, since it repeats information but the remainder should stay normal.

✗ **Casse** — même défaut : la portion barrée ne se réenroule pas, elle pousse le reste de la phrase au besoin ou déborde.

3.2 Équations — c’est ici que ça marche bien

Cas B.4 — équation inline

$$E=mc^2$$

△ **Fonctionne, mais...** — la barre apparaît enfin, mais box() transforme l’équation en boîte inline plate — elle perd sa position naturelle dans le texte (ici, seule sur sa ligne parce que testée isolément, mais elle ne se comporterait plus comme un vrai objet mathématique inline dans une phrase).

Cas B.5 — équation display courte, box() seul (perd son statut de bloc)

```
#strike-b($ E = m c^2 $)
```

Before. $E=mc^2$ After.

△ **Fonctionne, mais...** — la barre est là, mais l’équation n’est plus centrée sur sa propre ligne — elle est redevenue un élément **inline**, coincée entre « Before. » et « After. ». Il faut explicitement l’envelopper dans block() + align(center, ...) pour retrouver l’apparence normale d’une équation display (voir B.6).

Cas B.6 — équation display courte, block() + align(center, ...) en plus

```
#align(center, block(strike-b($ E = m c^2 $)))
```

$$E=mc^2$$

✓ **Fonctionne** — en rajoutant explicitement block() + align(center, ...), on retrouve l’apparence normale d’une équation display, avec une vraie barre.

Cas B.7 — équation display longue / large

```
#strike-b-centered($ P("y" | X) = "logit"^(-1)(beta_0 + beta_1 X_1 + beta_2 X_2 + beta_3 X_3) $)
```

$$P(y|X)=\text{logit}^{-1}(\beta_0+\beta_1X_1+\beta_2X_2+\beta_3X_3)$$

✓ **Fonctionne** — fonctionne quelle que soit la largeur — mais voir la mise en garde de la section 4 sur les colonnes étroites (une équation trop large pour sa colonne déborde de la même façon **avec ou sans** barre : ce n’est pas un problème créé par cette méthode.

Cas B.8 — équation display sur deux lignes, superposition naïve (une seule ligne)

```
#strike-b-centered($ a + b + c + d \ + e + f + g + h $)
```

$$\frac{a+b+c+d}{+e+f+g+h}$$

✗ **Casse** — la ligne unique se place à mi-hauteur **entre** les deux rangées de l’équation — ça ressemble à une barre de fraction, pas à un texte barré. Aucune des deux rangées n’est vraiment barrée.

Cas B.9 — équation display sur deux lignes, superposition « consciente » du nombre de lignes (rows: 2)

```
#align(center, block(strike-b-rows($ a + b + c + d \ + e + f + g + h $, rows: 2)))
```

$$\frac{-a+b+c+d}{+e+f+g+h}$$

△ **Fonctionne, mais...** — correct ici, **mais** demande de connaître (et de fournir explicitement) le nombre de rangées — rien dans Typst ne permet de le déduire automatiquement pour une équation quelconque. Si les rangées ont des hauteurs très différentes (une fraction sur une ligne, du texte plat sur l’autre), la répartition uniforme sz.height / rows peut mal aligner certaines barres.

Cas B.10 — un seul terme barré à l’intérieur d’une équation

```
$ beta_0 + #strike-b(beta_1 X_1$) + beta_2 X_2 $
```

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2$$

✓ **Fonctionne** — fonctionne bien : le terme ciblé est correctement barré, et comme il s’agit d’un terme **inline** au sein d’une plus grande équation (donc de toute façon jamais “réenroulé” mot à mot), le défaut de B.2/B.3 ne s’applique pas ici.

4 Méthode C — dispatch : texte natif, équation display en superposition

Idée : ne changer **que** le rendu des équations display, laisser le texte (et les équations inline) intégralement gérés par `strike()/underline()` natifs, qui fonctionnent déjà très bien pour eux. Détection via `body.func() == math.equation` and `body.at("block", default: false)`.

```
#let strike-smart(body) = {
  if type(body) == content and body.func() == math.equation
    and body.at("block", default: false) {
    strike-b-centered(body)
  } else {
    strike(body)
  }
}
```

Cas C.1 — texte seul (doit passer par la branche native)

~~The treatment has an effect on survival.~~

✓ **Fonctionne** — identique à A.1, comme attendu.

Cas C.2 — équation display seule (doit passer par la superposition)

~~$$E=mc^2$$~~

✓ **Fonctionne** — identique à B.6.

Cas C.3 — le cas difficile : texte + équation display + texte, EN UN SEUL appel

```
#strike-smart[was modeled as
$ P(x) = beta_0 + beta_1 X_["senior"] + beta_2 X_["length"] $
with $X$ the vector of covariates.]
```

~~was modeled as~~

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

~~with X the vector of covariates.~~

✗ **Casse** — `strike-smart` reçoit ici une SÉQUENCE (texte, équation, texte), pas une équation nue — la condition `body.func() == math.equation` est fausse pour la séquence entière, donc tout retombe dans la branche native, et l'équation n'est de nouveau pas barrée sur ses symboles. Un dispatch qui ne regarde que le nœud racine ne suffit pas dès que le contenu est mixte — exactement le cas réel de `#rep/#del` dans le package, jamais une équation toute seule.

Cas C.4 — dispatch récursif : on descend dans les séquences pour trouver les équations display

```
#let strike-smart-deep(body) = {
  if type(body) != content { body }
  else if body.func() == math.equation and body.at("block", default: false) {
    strike-b-centered(body)
  } else if body.func() == sequence {
    body.children.map(strike-smart-deep).sum()
  } else {
    strike(body)
  }
}
```

~~was modeled as~~

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

~~with X the vector of covariates.~~

△ **Fonctionne, mais...** — ça marche : le texte est barré nativement (reflow correct), l'équation display reçoit la superposition. **Mais** chaque fragment de texte entre deux équations est maintenant barré par un appel `strike()` **séparé** plutôt qu'un seul — à vérifier si ça laisse des micro-discontinuités visuelles sur des cas plus complexes (plusieurs équations proches, texte très court entre deux). Pas testé plus loin ici — à creuser si cette méthode est retenue.

5 Pistes tirées de l'issue GitHub `typst/typst#2200`

Le problème (« text decoration functions not working with inline equations ») est une issue ouverte et connue du cœur de Typst, pas une particularité de ce package. Extraction et test des pistes qui s'y trouvent, dans l'ordre où elles apportent quelque chose de nouveau par rapport aux méthodes A–C déjà testées ci-dessus.

5.1 `math.cancel` — une vraie décoration native, mais dans les maths seulement

Suggestion reçue en dehors de l'issue elle-même (conseil communautaire en marge) : `math.cancel`, une fonction native de Typst conçue pour « barrer un équation » dans une expression mathématique — mais rien n'empêche de l'utiliser pour barrer une fraction entière. Contrairement à `strike()`, elle opère à l'intérieur du moteur de rendu mathématique : elle barre les vrais glyphs (chiffres, lettres, opérateurs), pas seulement le texte entre guillemets.

Cas G.1 — terme isolé à l'intérieur d'une équation, angle par défaut

```
$a + cancel(b) + c$
```

$a + b + c$

✓ **Fonctionne** — une vraie diagonale traverse le glyphe « b » lui-même — jamais obtenu avec `strike()/underline()` (section 1).

Cas G.2 — équation entière, angle par défaut vs `angle: 90deg`

```
$ cancel(E = m c^2) $
$ cancel(E = m c^2, angle: #90deg) $
```

$E = mc^2$
 $E = mc^2$

△ **Fonctionne, mais...** — les deux « fonctionnent » (vrais glyphes barrés), mais l'angle par défaut trace une diagonale — la notation mathématique classique pour « ce facteur se simplifie », pas un signal de suppression. `angle: 90deg` (malgré le nom, donne une ligne quasi horizontale, pas verticale) est ce qui ressemble réellement à un texte barré.

Cas G.3 — équation display longue / large, `angle: 90deg`

$$P(y|X) = \text{logit}^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$$

✓ **Fonctionne** — une seule ligne continue traverse toute la largeur de l'équation, glyphes réels compris — exactement ce que `strike()` ne sait pas faire (case A.5).

Cas G.4 — équation display sur deux lignes, un `cancel()` par rangée

```
$ cancel(a + b + c + d, angle: #90deg) \ cancel(+ e + f + g + h, angle: #90deg) $
```

$a + b + c + d$
 $+ e + f + g + h$

✓ **Fonctionne** — chaque rangée est correctement barrée sur sa propre ligne — mais il faut envelopper **chaque** rangée séparément dans son propre `cancel()`, exactement la même contrainte que `rows`: en méthode B (case B.9) : rien ne permet à Typst de deviner automatiquement combien de rangées il y a.

Limite structurelle de `cancel()`, différente de celle de `strike()`. `cancel()` doit être appelé à l'intérieur d'une expression mathématique, sur du contenu mathématique — impossible de l'utiliser directement sur un passage qui mélange texte normal et équation en un seul appel, contrairement à `strike(...)` (case A.1–A.7) ou `text(fill: ...)` (méthode D, plus loin). Pour un usage réaliste dans `add/del`, il faudrait soit que l'auteur l'appelle lui-même **dans** chaque équation à marquer (perd l'automatisme d'un simple `#del(...)` autour de tout le passage), soit un dispatch qui descend jusqu'aux équations pour y injecter `cancel()` automatiquement — voir G.9/G.10 ci-dessous pour ce que ça donne en pratique.

5.2 `ul()` par `box + stroke`, sans `measure()` — mais seulement pour le soulignement

```
#let ul(content) = box(
  content,
  outset: (bottom: .1em),
  stroke: (bottom: .5pt),
)
```

Cas G.5 — `ul()` sur une équation inline

Before $e = mc^2$ after.

✓ **Fonctionne** — plus simple que la méthode B pour le soulignement : `box()` sait dessiner un trait sur son propre bord bas via `stroke`, sans jamais appeler `measure()` ni `place()` — Typst calcule lui-même la largeur de la boîte. Mais un bord de boîte ne peut se poser qu'en haut/bas/gauche/droite, jamais à mi-hauteur : cette simplification ne marche que pour souligner, pas pour barrer.

Cas G.6 — `ul()` sur un texte long, en paragraphe — révérifie le bug de la méthode B avec un code différent

This is a fairly long sentence that should wrap across more than one line when placed inside a base of normal width. to check whether the underline reflows correctly word by word.

X **Casse** — même défaut que B.2, avec une implémentation entièrement différente (pas de `measure()`, pas de `context`) : la cause profonde n'est donc pas « mesurer coûte cher » ou « le calcul de taille est fragile », c'est `box()` lui-même qui rend son contenu atomique dès qu'on l'utilise pour y accrocher un trait. Confirme que le problème de M1 (CLAUDE.md §6bis) est structurel à `box`, pas à un détail d'implémentation qu'on aurait pu éviter.

5.3 `st()` de l'issue — décalage fixe vs proportionnel

```
#let st(content) = context {
  let width = measure(content).width
  box(
    #content
    #place(dy: -3pt, line(stroke: .5pt, length: width))
  )
}
```

Cas G.7 — `st()` (décalage fixe -3pt) comparé à `strike-b` (méthode B, proportionnel `height/2`), à trois tailles de police plus un contenu haut (fraction)

10pt : $e = mc^2$ vs $e = mc^2$

8pt : $e = mc^2$ vs $e = mc^2$

18pt : $e = mc^2$ vs $e = mc^2$

fraction (10pt) : $\frac{a}{b}$ vs $\frac{a}{b}$

△ **Fonctionne, mais...** — à 10pt les deux se ressemblent (l'exemple de l'issue est visiblement calibré pour une taille proche de celle-ci). Dès qu'on s'en écarte, `st()` se dérègle nettement : à 8pt la barre remonte trop haut, à 18pt elle tombe bien en dessous du centre (proche de la ligne de base), et sur une fraction elle finit collée à la barre de fraction plutôt que centrée sur la hauteur totale. Le calcul proportionnel déjà utilisé dans `strike-b`/méthode B (`sz.height / 2`) reste correct dans tous les cas — ce n'est pas une régression à corriger, juste une confirmation que le choix déjà fait était le bon.

5.4 La piste la plus utile : une show rule scopée pour éviter le dispatch manuel

Dans l'issue, tedzards509 puis mariansam contournent la limite en ciblant l'intérieur d'un élément avec une show rule posée juste avant de le réémettre :

```
#show highlight: it => {
  show math.equation: box.with(fill: it.fill)
  it
}
```

Idee : une show rule posée sur `math.equation`, déclarée **dans la portée** du traitement d'un autre élément, s'applique à toutes les équations que Typst rencontre en construisant cet élément — y compris celles nichées dans une séquence texte + équation + texte. C'est exactement le problème non résolu en méthode C (cases C.3/C.4) : au lieu d'écrire soi-même une récursion qui reconnaît une « séquence » et qui doit être tenue à jour à la main, on laisse le moteur de mise en page de Typst faire cette traversée, puisqu'il la fait de toute façon.

```
#let strike-g(body) = {
  show math.equation.where(block: true): eq => strike-b-centered(eq)
  strike(body)
}
```

Cas G.8 — le cas difficile de C.3/D.9 : texte + équation display + texte, en un seul appel, sans aucune récursion écrite à la main

was-modeled-as

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

with X the vector of covariates.

✓ **Fonctionne** — identique visuellement à C.4, mais `strike-g` ne contient aucune logique de parcours d'arbre — pas de `repr(body.func())` == "sequence", pas de `.children.map(...)`. Typst descend lui-même dans la séquence pour appliquer la show rule à l'équation qu'il y trouve. Plus court, et plus robuste face à des structures que la récursion manuelle de C.4 ne connaît pas (une équation nichée dans une table, une liste, etc. — jamais testé explicitement mais la mécanique ne dépend pas du type de conteneur, contrairement à C.4).

Cas G.9 — tentative : remplacer la ligne de superposition par `math.cancel` dans cette même show rule

```
#let strike-g-cancel(body) = {
  show math.equation.where(block: true): eq =>
    align(center, block(math.cancel(eq.body, angle: 90deg)))
  show math.equation.where(block: false): eq =>
    math.cancel(eq.body, angle: 90deg)
  strike(body)
}
```

X **Casse** — `error: maximum show rule depth exceeded`. Réémettre du contenu mathématique à l'intérieur d'une show rule qui cible `math.equation` retombe sur une équation (re)générée par Typst lui-même en interne (auto-mathification), qui matche à nouveau la même règle — boucle infinie détectée et refusée par le compilateur. Essayé aussi en reconstruisant explicitement via `eq.func()(math.cancel(eq.body, ...), ..champs)` avec un `show math.equation: it => it` posé en garde juste avant : échoue exactement pareil. Contrairement à G.8 (qui réémet une simple superposition `box/place`, jamais un `math.equation`), toute tentative de renvoyer quelque chose qui **redevient** une équation depuis l'intérieur d'une show rule sur `math.equation` semble structurellement piégée dans cette version de Typst. Non résolu, pas de contournement propre trouvé — signalé ici comme un mur réel plutôt que creusé davantage.

5.5 G.8 a un angle mort : les équations inline — peut-on le combler ?

G.8 ne cible que `math.equation.where(block: true)` : une équation inline (with `$x$` the vector...) retombe dans la branche `strike()` native et n'est donc, comme partout ailleurs dans ce document, pas visuellement barrée sur ses symboles. Rien n'empêche d'ajouter une seconde règle pour `block: false`, avec la version non centrée de la superposition de `strike-b`, pas `strike-b-centered`) :

```
#let strike-g2(body) = {
  show math.equation.where(block: true): eq => strike-b-centered(eq)
  show math.equation.where(block: false): eq => strike-b(eq)
  strike(body)
}
```

Cas G.10 — le même cas que G.8, avec la règle `block: false` en plus

was-modeled-as

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

with X the vector of covariates.

✓ **Fonctionne** — cette fois le x inline est bien barré aussi — sur le rendu de ce cas précis, G.10 est visuellement complet.

Cas G.11 — plusieurs équations inline courtes dans une phrase qui doit se réenrouler

The estimate $\hat{\beta}_1$ was compared to $\hat{\beta}_2$ and to a null value of 0 across a fairly long sentence that should still wrap normally onto more than one line, to check whether many small boxed equations disturb text reflow the way a single big box would.

✓ **Fonctionne** — le reflow reste correct : chaque équation inline est une **petite** boîte isolée, pas toute la phrase — elle se comporte comme un mot un peu large parmi d'autres, pas comme le bloc atomique de B.2/B.3.

Cas G.12 — le vrai défaut, révélé par une équation inline longue dans une colonne étroite

// page large de 8cm, pour forcer la question à se poser
Filler words before the equation \$a + b + c + d + e + f + g + h + i + j + k\$
and filler words after to see what happens at the margin.

Avec `strike()` natif (aucune superposition) :

Filler words before the equation $a + b + c + d + e + f + g + h + i + j + k$ and filler words after to see what happens at the margin.

Avec `strike-g2` (l'équation inline passe par `strike-b`) :

Filler words before the equation $a + b + c + d + e + f + g + h + i + j + k$ and filler words after to see what happens at the margin.

X **Casse** — `strike()` natif laisse Typst **casser l'équation elle-même** au niveau de ses +, exactement comme n'importe quel texte qui se réenroule normalement. `strike-g2`, en la tenant plus sur la ligne courante et se retrouve rejetée entière sur la ligne suivante, laissant un trou visible avant elle. C'est le même défaut de fond que celui déjà rejeté en M1/case B.2 — juste réduit à l'échelle d'une équation inline au lieu de tout un paragraphe, donc moins fréquent (il ne se déclenche que si une équation inline est elle-même assez longue pour ne plus tenir), mais pas éliminé. Une équation inline courte (un symbole, un sous-script) ne le déclenche jamais ; une équation inline qui s'approche de la largeur d'une colonne (le genre de choses déjà repéré comme risqué dans `charged-ieee`, §6quinquies) le déclenche.

Donc G.10/G.11 ne sont pas un « G.8 en mieux » sans réserve : ils corrigent l'angle mort visuel sur les équations inline courtes (le cas le plus courant en pratique — un symbole seul, un sous-script), au prix de réintroduire, seulement pour les équations inline **longues**, exactement le compromis que ce document a déjà écarté pour le texte de prose. La méthode D (teinte) n'a ce problème à aucune échelle (case D.4, une équation inline colorée reste un vrai élément inline, quelle que soit sa longueur — aucune `box`, donc rien à casser) : c'est un point de plus en sa faveur, pas contre elle.

5.6 Injection automatique de `math.cancel` — dispatch manuel plutôt que show rule

G.9 a échoué parce qu'une **show rule** qui réémet du contenu redevenant une équation retombe sur elle-même. Rien n'empêche en revanche d'injecter `cancel()` via le **même dispatch manuel récursif** que la méthode C (cases C.4/G.8) — une reconstruction ponctuelle, appelée une fois, pas une règle qui reste active pour le reste du document :

```
#let mark-eq-cancel(eq) = {
  let f = eq.fields()
  let b = f.remove("body")
  let lbl = f.remove("label", default: none)
  let new-eq = eq.func()(math.cancel(b, angle: 90deg), ..f)
  if lbl != none { #new-eq#lbl } else { new-eq }
}

#let strike-cancel-deep(body) = {
  if type(body) != content { body }
  else if body.func() == math.equation {
    mark-eq-cancel(body)
  } else if repr(body.func()) == "sequence" {
    body.children.map(strike-cancel-deep).sum(default: [])
  } else {
    strike(body)
  }
}
```

Sur `f.remove("label", ...)` puis `#new-eq#lbl`. Reconstruire via `eq.func()(..f)` avec `label` encore présent dans `f` échoue (unexpected argument: label — un label ne se passe pas comme paramètre nommé du constructeur). Il faut le retirer des champs, puis le raccrocher **après coup** en le plaçant juste derrière le nouveau contenu dans une séquence (`#new-eq#lbl`) — la même syntaxe que `figure(...)` <label> en markup, ici obtenue par construction. `new-eq + lbl` échoue direct (« cannot add content and label »).

Cas H.1 — mixte texte + équation display + texte, en un seul appel — cette fois sans passer par une show rule

was-modeled-as

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

with X the vector of covariates.

✓ **Fonctionne** — fonctionne, sans le mur de G.9 : le dispatch récursif reconstruit une fois, il ne se réenregistre jamais comme show rule active — rien à matcher deux fois.

Cas H.2 — équation étiquetée : la numérotation et `#ref` survivent-elles à la reconstruction ?

$$E = mc^2 \tag{1}$$

See Equation 1.

✓ **Fonctionne** — oui — numérotation et renvoi résolvent normalement une fois le label correctement retiré puis raccroché (voir la remarque ci-dessus).

Cas H.3 — équation display sur deux lignes (un seul corps, coupé par `\`) : `cancel()` voit-il les deux rangées ?

$$\frac{a + b + c + d}{+ e + f + g + h}$$

X **Casse** — non : une seule ligne diagonale traverse la boîte englobante des **deux** rangées à la fois, exactement entre elles — ça ressemble à une barre de fraction, pas à un texte barré (même défaut que la case B.8 de la méthode B). `math.cancel` calcule sa ligne sur la boîte englobante de son argument, sans regarder s'il contient un retour à la ligne interne — passer tout le corps d'un coup, sans le scinder par rangée, ne fonctionne pas mieux ici qu'avec la superposition maison.

Cas H.4 — LE test décisif : la même équation inline longue que G.12, dans la même colonne étroite, automatiquement marquée par `cancel()` plutôt que par une boîte

Filler words before the equation $a + b + c + d + e + f + g + h + i + j + k$ and filler words after to see what happens at the margin.

X **Casse** — même défaut que G.12, pour une raison différente et plus surprenante : `strike-cancel-deep` ne pose ici **aucune** `box/place` (juste `math.cancel()`, une fonction native) — et pourtant l'équation redevient tout aussi insécable, débordant du même bloc de la même façon. `cancel()` semble donc, en interne, traiter tout son argument comme une unité de mise en page atomique pour pouvoir y tracer sa diagonale — exactement le même compromis que la superposition maison, mais **intégré à la fonction native elle-même**, pas introduit par le code du package. Ce n'est pas un défaut de `strike-cancel-deep` : c'est une limite de `cancel()` découverte en le testant à cette échelle, qu'aucune méthode d'injection (show rule ou dispatch manuel) ne peut éviter.

Cas H.5 — plusieurs équations inline courtes dans une phrase qui se réenroule, pour vérifier que ce n'est pas un problème général en usage courant

The estimate $\hat{\beta}_1$ was compared to $\hat{\beta}_2$ and to a null value of 0 across a fairly long sentence that should still wrap normally onto more than one line.

✓ **Fonctionne** — correct — comme en G.11, le défaut ne se déclenche que si une équation inline individuelle est elle-même trop longue pour la ligne, pas simplement parce qu'il y en a plusieurs.

Conclusion sur l'injection automatique de `cancel()`. Le dispatch manuel (H) évite bien le mur de la show rule (G.9) — techniquement faisable. Mais il hérite de deux défauts propres à `math.cancel` lui-même, indépendants de la méthode d'injection : (1) une équation multi-rangées reçoit une seule ligne traversant les deux rangées à la fois (H.3, identique à B.8) — non résolu automatiquement, demanderait de détecter et scinder par rangée avant d'appeler `cancel()`, pas tenté ici ; (2) une équation inline assez longue perd sa capacité à se couper à ses propres + (H.4) — le même compromis que la superposition (G.12), cette fois causé par `cancel()` lui-même, pas par une `box` du package. `math.cancel` reste une bonne option pour une équation **courte**, appelée à la main sur un cas ponctuel ; comme mécanisme **automatique** et général pour `del()`, il n'élimine aucun des deux problèmes déjà identifiés pour les approches par superposition — seulement leur cause change.

6 Méthode D — teinte de couleur seule, sans aucune décoration

Suite à la question : **et si le problème, c'était justement de vouloir tracer une ligne ?** `text(fill: ...)` n'est pas une décoration attachée à des runs de texte comme `strike/underline/highlight` — c'est la couleur avec laquelle **n'importe quel glyphe** est dessiné, texte ou math. Elle ne nécessite ni mesure, ni boîte, ni ligne placée à la main — donc aucun des problèmes des méthodes B/C.

```
#let strike-d(body) = text(fill: gray, body)
```

6.1 Texte

Cas D.1 — texte court

The treatment has an effect on survival.

✓ **Fonctionne** —

Cas D.2 — texte long, entièrement grisé

The treatment was associated with a clinically meaningful reduction in the primary composite outcome, driven mainly by a reduction in cardiovascular death rather than non-fatal myocardial infarction, though the confidence interval remained fairly wide given the modest sample size.

✓ **Fonctionne** — reflow parfait, comme la méthode A — normal, on ne touche à aucune mesure ni boîte.

Cas D.3 — texte long, partiellement grisé

This entire opening clause of the sentence is what we want removed, since it repeats information already given earlier and adds nothing new for the reader to consider here today, but the remainder should stay completely normal.

✓ **Fonctionne** —

6.2 Équations

Cas D.4 — équation inline

The relation $e = mc^2$ holds.

✓ **Fonctionne** — reste un vrai élément inline, contrairement à B.4 — on n'a pas touché à box.

Cas D.5 — équation display courte

$$E = mc^2$$

✓ **Fonctionne** — reste centrée, sur sa propre ligne, comme une équation display normale — on n'a rien changé à sa mise en page.

Cas D.6 — équation display longue / large

$$P(y \mid X) = \text{logit}^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$$

✓ **Fonctionne** —

Cas D.7 — équation display sur deux lignes

$$\begin{array}{c} a + b + c + d \\ + e + f + g + h \end{array}$$

✓ **Fonctionne** — **aucun réglage rows: nécessaire** — la couleur s'applique aux deux rangées automatiquement, sans qu'on ait besoin de savoir combien il y en a.

Cas D.8 — un seul terme grisé à l'intérieur d'une équation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2$$

✓ **Fonctionne** —

Cas D.9 — le cas difficile de C.3 : texte + équation display + texte, EN UN SEUL appel, sans aucun dispatch

was modeled as

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}} + \beta_2 X_{\text{length}}$$

with X the vector of covariates.

✓ **Fonctionne** — **fonctionne du premier coup**, sans la variante récursive qu'il a fallu construire en C.4 — `text(fill: ...)` n'a pas besoin de savoir ce qu'il y a dans `body`, contrairement à un dispatch qui doit reconnaître une équation display pour la traiter différemment.

7 Le vrai problème : add et del sont aujourd’hui indiscernables sur les maths

En creusant **pourquoi** une barre semblait nécessaire, le vrai enjeu apparaît : dans le package aujourd’hui, add et del appliquent tous les deux **la même couleur** (celle du relecteur) et ne se distinguent que par la décoration (underline vs strike). Sur du texte, ça marche. Sur des maths, la décoration est invisible (section 1) — donc un ajout et une suppression de maths sont **visuellement identiques** aujourd’hui, pas seulement “pas barrées”.

Cas E.1 — aujourd’hui : ajout et suppression de la même expression, côte à côte

Ajout : $\beta_1 X_{\text{senior}}$ — Suppression : $\beta_1 X_{\text{senior}}$

✗ **Casse** — strictement impossible à distinguer sans zoomer sur le sous-script « senior » (le seul bout de vrai texte).

La méthode D suggère la vraie correction : ne pas chercher à barrer les maths, mais donner à del une teinte **distincte** de celle d’add — moins saturée, plus sombre — qui, elle, s’applique uniformément à tout, texte et maths, sans aucune décoration :

```
#let del-color(c) = c.desaturate(60%).darken(15%)
```

Cas E.2 — même expression, avec la couleur de suppression assourdie

Ajout : $\beta_1 X_{\text{senior}}$ — Suppression : $\beta_1 X_{\text{senior}}$

✓ **Fonctionne** — distinguable au premier coup d’œil, sans avoir besoin de zoomer ni de chercher un sous-script.

Cas E.3 — même chose avec un autre relecteur (bleu), pour vérifier qu’on garde l’identité du relecteur

Ajout : $\beta_1 X_{\text{senior}}$ — Suppression : $\beta_1 X_{\text{senior}}$

✓ **Fonctionne** — la suppression reste reconnaissable comme venant du même relecteur (toujours dans les bleus), juste assourdie.

Cas E.4 — sur un passage mixte complet : ajout souligné+coloré vs suppression barrée+assourdie

Ajout : was modeled as

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}}$$

with X the vector.

Suppression : ~~was modeled as~~

$$P(x) = \beta_0 + \beta_1 X_{\text{senior}}$$

~~with X the vector.~~

✓ **Fonctionne** — le texte cumule les deux signaux (couleur + décoration) ; les maths n’ont que la couleur, mais ça suffit à les distinguer clairement l’une de l’autre. Aucune mesure, aucune boîte, aucun dispatch par type de contenu — on garde strike/underline tels quels pour le texte (gratuits, déjà parfaits) et on ajoute juste une teinte différente pour del.

Sur le choix exact de la teinte : desaturate(60%).darken(15%) n’est qu’un premier essai. D’autres formules testées en aparté (desaturate(50%).lighten(15%), transparentize(45%), un simple gray fixe non lié au relecteur, un mélange color.mix avec du gris) donnent des résultats assez proches, souvent **trop** pâles à partir d’un rouge très saturé. Le réglage fin (à quel point assourdir, éclaircir ou assombrir) reste à ajuster visuellement une fois la direction générale validée — ce n’est pas un choix technique bloquant.

8 Un problème encore plus extrême : les figures supprimées

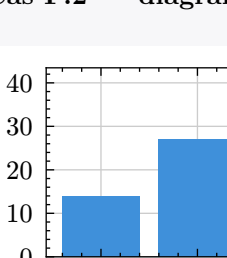
Aujourd'hui (`deleted()`/`del()` sur un `figure(...)`), seule la **légende** se barre — c'est du vrai texte, `strike()` la traite très bien. Le corps de la figure (un `rect`, une `image`, un `graphique`) ne contient généralement **aucun texte du tout** : ce n'est pas seulement "la décoration ne l'atteint pas comme pour les maths", c'est qu'il n'y a rien de textuel à décorer par principe. `text(fill: ...)` (méthode D) ne peut pas non plus aider directement : la couleur d'un `rect` ou d'une `image` n'est pas **héritée** de la couleur de texte ambiante, elle est fixée dans ses propres attributs (`fill`).

Deux familles d'approches possibles : **recolorer le contenu lui-même** (reconstruire ses attributs de couleur), ou **superposer** quelque chose par-dessus sans toucher au contenu. Comme une figure est déjà un bloc autonome (elle ne se réenroule jamais mot à mot comme du texte), le risque de la méthode B (mesure + boîte qui casse le reflow) **ne s'applique pas ici** — une figure supprimée n'a jamais besoin de se réenrouler.

8.1 Recolorer le contenu — fragile, à éviter

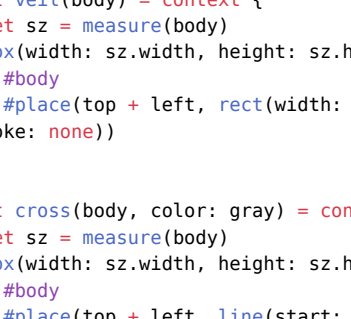
```
#let regrey rect(node) = {
  if type(node) == content and node.func() == rect {
    let f = node.fields()
    f.insert("fill", gray)
    f.insert(...f)
  } else {
    node // ne sait rien faire d'autre : silencieusement inchangé
  }
}
```

Cas F.1 — `rect` simple : ça marche, parce qu'on a explicitement prévu ce cas



△ **Fonctionne, mais...** — fonctionne, mais seulement parce que la fonction connaît spécifiquement `rect`.

Cas F.2 — `diagramme titlaq` : silencieusement rien ne se passe

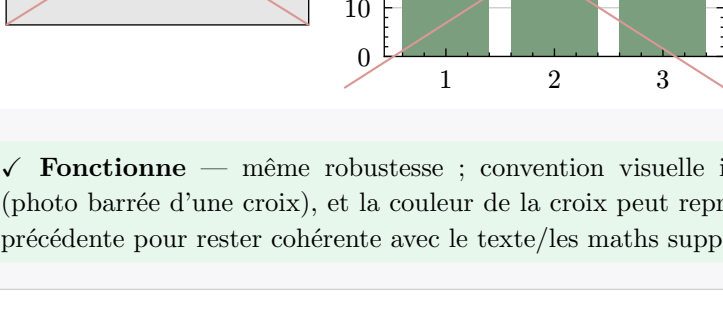


X **Casse** — aucune erreur, mais aucun changement non plus — `titlaq` construit ses graphiques avec des éléments internes (`etembic`) que la fonction ne reconnaît pas. Même limite de fond que `strip-labels` avec ce même package (`CLAUDE.md §6quatervicies`) : une approche par reconstruction doit connaître **chaque** package de contenu possible à l'avance, ce qui est structurellement sans fin dès qu'un vrai package de tracé ou de dessin entre en jeu.

8.2 Superposition — robuste, ne regarde jamais ce qu'il y a dessous

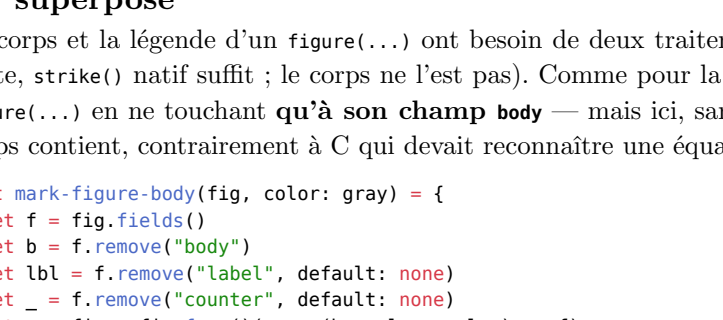
```
#let veil(body) = context {
  let sz = measure(body)
  box(width: sz.width, height: sz.height) {
    #body
    #place(top + left, rect(width: sz.width, height: sz.height, fill: white.transparentize(35%),
  stroke: none))
  }
}
```

Cas F.3 — voile translucide, sur un `rect` placeholder ET sur un vrai diagramme `titlaq`



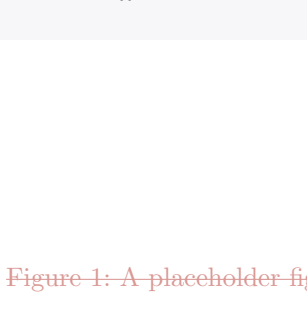
✓ **Fonctionne** — marche sur les deux sans exception — `veil` ne regarde jamais ce qu'il y a dans `body`, elle mesure sa taille et pose un rectangle par-dessus. Aucun risque de type de contenu non reconnu.

Cas F.4 — croix diagonale, mêmes deux figures



✓ **Fonctionne** — même robustesse ; convention visuelle immédiatement reconnaissable (photo barrée d'une croix), et la couleur de la croix peut reprendre `del-color()` de la section précédente pour rester cohérente avec le texte/les maths supprimés.

Cas F.5 — voile + croix combinés



✓ **Fonctionne** — plus doux visuellement que la croix seule ; question de goût plus que de robustesse.

8.3 Assemblage réaliste : légende barrée nativement, corps de figure superposé

Le corps et la légende d'un `figure(...)` ont besoin de deux traitements différents (la légende est du texte, `strike()` natif suffit ; le corps ne l'est pas). Comme pour la méthode C, il faut reconstruire le `figure(...)` en ne touchant **qu'à son champ `body`** — mais ici, sans avoir besoin de savoir ce que ce corps contient, contrairement à C qui devait reconnaître une équation.

```
#let mark-figure-body(fig, color: gray) = {
  let f = fig.fields()
  let b = f.remove("body")
  let lbl = f.remove("label", default: none)
  let _ = f.remove("counter", default: none)
  let new_fig = fig.func()(cross(b, color: color), ..f)
  if lbl != none { [new_fig|lbl] } else { new_fig }
}
```

Corrigé par rapport à une première version. Reconstruire directement via `fig.func(..f)` sans retirer `label` ni `counter` échoue : `label` n'est pas un paramètre nommé valide du constructeur `figure()` (unexpected argument: label, même piège que pour les équations, H.2) et `counter` est un champ **synthétisé par Typst à la préparation de l'élément** — absent quand on construit une figure à la main (cas F.6-F.8 ci-dessus, jamais étiquetées, jamais découvert le problème), mais présent dès qu'on récupère une figure réelle qui a déjà transité par la mise en page (cas F.9 et « assemblage dans un `del()` plus large » ci-dessous). Les deux se retirent des champs avant reconstruction ; le `label` se raccroche après coup exactement comme en H.2 (`!#new_fig|lbl`), pour que la numérotation et un `@ref` externe continuent de fonctionner.

Cas F.6 — figure `rect`, corps croisé + légende barrée nativement, le tout passé à un seul `strike()`

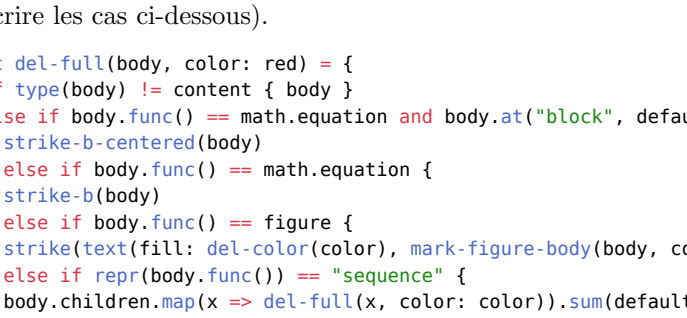


Figure-1: A placeholder-figure-caption, long enough to wrap on more than one line to check reflow.

✓ **Fonctionne** — la légende se réenroule et se barre normalement (texte réel) ; le corps est croisé, dans la même teinte assourdie que la légende — cohérent avec la méthode D/E pour le texte et les maths du même passage.

Cas F.7 — même chose avec un vrai graphique `titlaq`

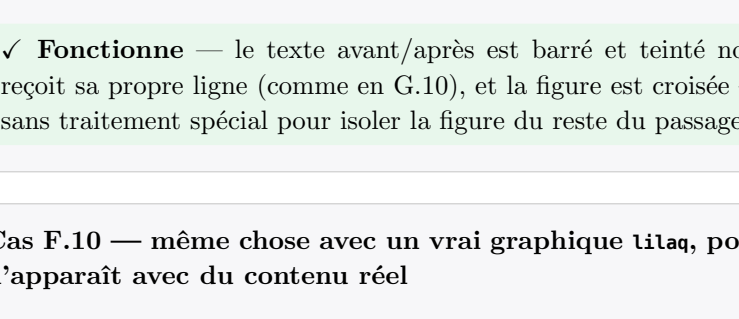


Figure-2: A titlaq-bar-chart-caption.

✓ **Fonctionne** — identique, sans aucun crash ni recours à une reconstruction interne du graphique — contrairement à `strip-labels`, `mark-figure-body` ne descend jamais dans le corps, elle le traite comme une boîte opaque.

Cas F.8 — figure `table` (données, pas un dessin) : le voile laisse aussi `strike()` natif atteindre le texte des cellules

1	2	3
4	5	6

Figure-3: A data-table-caption.

✓ **Fonctionne** — bonus : les cellules, étant du vrai texte, se barrent nativement **en plus** d'être voilées/croisées — les deux effets se cumulent sans conflit.

Pourquoi ceci n'a pas le problème de la méthode B. La superposition (`veil/cross`) utilise la même mécanique `measure()` + `box` + `place()` que le « `mark-line` » abandonné en M1 — mais elle n'est appliquée **qu'à des figures**, jamais à du texte de prose qui doit se réenrouler. Une figure est, par nature, déjà un bloc de taille fixe : la mesurer et l'envelopper dans une boîte de cette même taille ne change rien à son comportement. C'est précisément le cas d'usage pour lequel cette technique est sûre — le problème de M1 venait de l'avoir appliquée **aussi** à du texte, pas de la technique elle-même.

8.4 Cas réaliste : la figure n'est jamais seule dans un `del()`

Question posée directement : dans l'usage réel du package, une figure supprimée n'est jamais passée seule à `mark-figure-body` — elle est **nichée** dans un `del()` qui couvre aussi du texte avant et après (le motif de tous les exemples réels, `examples/fridge-study`, `examples/emoji-email`). Est-ce que la croix continue de s'appliquer correctement à la figure dans ce contexte plus large, avec le reste du passage barré/teinté normalement autour ? Reproduit avec le dispatch récursif déjà validé en H (pas de show rule ici non plus : comme `math.cancel` en G.9, `mark-figure-body` reconstruit un **nouveau** `figure(...)`, donc un `show rule` sur `figure` retomberait sur elle-même — même piège, testé et confirmé avant d'écrire les cas ci-dessous).

```
#let del-full(body, color: red) = {
  if type(body) != content { body }
  else if body.func() == math.equation and body.at("block", default: false) {
    strike-b-centered(body)
  } else if body.func() == math.equation {
    strike-b(body)
  } else if body.func() == figure {
    strike(text(fill: del-color(color), mark-figure-body(body, color: del-color(color))))
  } else if repr(body.func()) == "sequence" {
    body.children.map(x => del-full(x, color: color)).sum(default: [])
  } else {
    strike(text(fill: del-color(color), body))
  }
}
```

Cas F.9 — figure `rect` avec texte avant/après ET une équation inline, dans un seul `appel`

This introductory sentence sets up the context before the figure and should be struck and tinted normally.

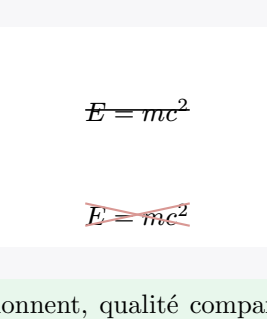


Figure-4: A placeholder-figure-caption, long enough to check that it wraps and strikes correctly.

This closing sentence comes after the figure and should also be struck and tinted, with an inline equation $\frac{a}{b}$ thrown in too.

✓ **Fonctionne** — le texte avant/après est barré et teinté normalement, l'équation inline reçoit sa propre ligne (comme en G.10), et la figure est croisée — le tout dans un seul `appel`, sans traitement spécial pour isoler la figure du reste du passage.

Cas F.10 — même chose avec un vrai graphique `titlaq`, pour vérifier qu'aucun conflit n'apparaît avec du contenu réel

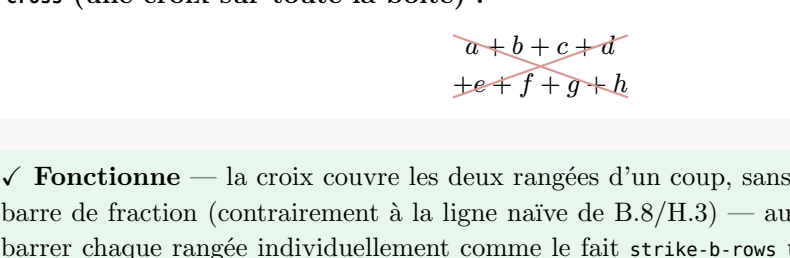


Figure-5: A real titlaq-bar-chart-caption, describing the now-removed analysis.

And a trailing sentence after the chart, to confirm reflow continues normally afterward.

✓ **Fonctionne** — identique — `mark-figure-body` ne regarde jamais l'intérieur de `body`, donc les labels internes de `titlaq/etembic` (§6quatervicies) ne sont jamais en jeu ici, contrairement à `strip-labels/pinpoint` (excerpt: true).

Cas F.11 — figure étiquetée dans ce même contexte élargi : numérotation et `@ref` externe au `del()`

Text before a labeled figure.

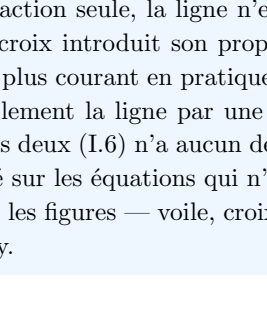


Figure-6: A labeled figure-caption.

Text after—Reference from outside the deletion: Figure 6.

✓ **Fonctionne** — la figure garde son vrai numéro de séquence et reste référencée depuis **en dehors** du passage supprimé — exactement le cas réel (§6quinquies : un passage supprimé peut très bien être cité ailleurs dans le manuscrit).

Réponse à la question posée. Oui : une figure supprimée au milieu d'un `del()` plus large (texte avant, plus nettement que prévu : sur une fraction seule, la ligne n'est pas juste « difficile à voir », elle est **totale**ment invisible (I.1). Mais la croix introduit son propre défaut symétrique sur les équations très larges et peu hautes (I.4, le cas le plus courant en pratique pour une équation `display` d'une seule ligne) — remplacer purement et simplement la ligne par une croix échangerait un défaut contre un autre, pas un progrès net. Combiner les deux (I.6) n'a aucun des deux défauts dans les cas testés ici — au prix d'un rendu un peu plus chargé sur les figures — voile, croix ou les deux se posent donc maintenant à l'identique pour les équations `display`.

9 Croix sur les équations bloc — comme pour les figures

Question posée directement : la ligne à mi-hauteur (`strike-b`, méthode B/G/H) peut-elle devenir invisible quand elle coïncide avec une barre déjà présente dans l'équation elle-même — une fraction, typiquement ? Est-ce qu'une croix diagonale (`cross`, déjà utilisée pour les figures en méthode F) réglerait ça pour les équations `display` (jamais les équations inline — une croix sur un simple symbole isolé au fil du texte serait disproportionnée, et l'inline garde de toute façon le défaut de réenroulement déjà documenté en G.12/H.4, sans rapport avec la forme de la marque). `cross` était déjà défini pour les figures (section 7) ; il suffit de l'appliquer à `body` directement, sans passer par `mark-figure-body` (pas de champs `label/counter` à gérer ici, contrairement à un vrai `figure(...)` — une équation qu'on marque ainsi reste elle-même inchangée, on colle juste la croix par-dessus, on ne la reconstruit pas).

```
#let cross-centered(body, color: gray) = align(center, block(cross(body, color: color)))
```

Cas I.1 — le cas décisif : une fraction seule, isolée — le pire cas pour la ligne

strike-b :

$$\frac{a}{b}$$

cross :

$$\frac{a}{b}$$

X **Casse** — confirmé — avec `strike-b`, la ligne à mi-hauteur de la boîte tombe exactement sur la barre de fraction déjà là : le résultat est visuellement **indiscernable** d'une fraction non marquée, aucune trace de suppression n'est visible. Avec `cross`, la diagonale traverse numérateur et dénominateur à des hauteurs différentes de la barre de fraction : les deux restent visuellement distincts, la suppression est repérable au premier coup d'œil.

Cas I.2 — même problème, fraction noyée dans une équation plus large

strike-b :

$$\frac{a}{b} + c$$

cross :

$$\frac{a}{b} + c$$

✓ **Fonctionne** — moins extrême qu'en I.1 (le reste de l'équation est bien barré par la ligne), mais la partie « a/b » reste ambiguë avec `strike-b` — avec `cross`, toute l'expression, fraction comprise, est marquée sans ambiguïté.

Cas I.3 — équation simple, sans fraction ni barre interne

strike-b :

$$E=mc^2$$

cross :

$$E=\overline{mc^2}$$

✓ **Fonctionne** — les deux fonctionnent, qualité comparable — la croix n'apporte rien de plus ici, mais ne casse rien non plus.

Cas I.4 — le revers de la médaille : une équation longue et large, peu haute

strike-b :

$$P(y|X) = \logit^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$$

cross :

$$\overline{P(y|X)} = \logit^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$$

X **Casse** — nouveau défaut, propre à `cross` : quand la boîte est beaucoup plus large que haute (le cas le plus fréquent pour une équation `display` d'une seule ligne), les deux diagonales d'un coin à l'autre sont presque plates — elles se rapprochent visuellement de deux lignes quasi parallèles collées l'une à l'autre plutôt que d'une croix reconnaissable. Moins clair qu'une simple ligne ici.

Cas I.5 — équation sur deux rangées : la croix évite-elle l'effet « barre de fraction » de la case B.8/H.3 ?

strike-b-centered naïf (une seule ligne entre les deux rangées, cf. B.8/H.3) :

$$\frac{a+b+c+d}{+e+f+g+h}$$

cross (une croix sur toute la boîte) :

$$\frac{a+b+c+d}{+e+f+g+h}$$

✓ **Fonctionne** — la croix couvre les deux rangées d'un coup, sans jamais ressembler à une barre de fraction (contrairement à la ligne naïve de B.8/H.3) — au prix de ne pas non plus barrer chaque rangée individuellement comme le fait `strike-b-rows` une fois qu'on lui précise `rows: 2`. Un compromis raisonnable si l'on ne veut pas avoir à détecter le nombre de rangées à l'avance.

Cas I.6 — peut-on avoir les deux à la fois : ligne + croix superposées ?

```
#let line-and-cross(body, color: gray) = context {
  let sz = measure(body)
  box(width: sz.width, height: sz.height) {
    #body
    #place(top + left, dy: sz.height / 2, line(length: sz.width, stroke: 0.6pt + color))
    #place(top + left, line(start: (0pt, 0pt), end: (sz.width, sz.height), stroke: 0.6pt + color))
    #place(top + left, line(start: (0pt, sz.height), end: (sz.width, 0pt), stroke: 0.6pt + color))
  }
}
```

fraction seule (I.1) :

$$\frac{a}{b}$$

équation large (I.4) :

$$\overline{P(y|X)} = \logit^{-1}(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3)$$

équation simple :

$$\overline{E=mc^2}$$

✓ **Fonctionne** — règle les deux problèmes en même temps : la ligne garantit un signal horizontal net même sur une équation large (I.4) ; les diagonales garantissent qu'une fraction ne peut plus se confondre avec une équation non marquée (I.1). Légèrement plus chargé visuellement sur une équation simple (trois traits au lieu d'un ou deux), mais jamais ambigu.

Réponse à la question posée. Oui, la croix règle bien le problème identifié — et le confirme même plus nettement que prévu : sur une fraction seule, la ligne n'est pas juste « difficile à voir », elle est **totale**ment invisible (I.1). Mais la croix introduit son propre défaut symétrique sur les équations très larges et peu hautes (I.4, le cas le plus courant en pratique pour une équation `display` d'une seule ligne) — remplacer purement et simplement la ligne par une croix échangerait un défaut contre un autre, pas un progrès net. Combiner les deux (I.6) n'a aucun des deux défauts dans les cas testés ici — au prix d'un rendu un peu plus chargé sur les figures — voile, croix ou les deux se posent donc maintenant à l'identique pour les équations `display`.

10 Récapitulatif

Cas	A (native)	B (over-lay)	C (dis-patch)	D (teinte)
Texte court/long, barré entier	✓	X (déborde)	✓ (= A)	✓
Texte partiellement barré	✓	X	✓ (= A)	✓
Équation inline	X	△ (perd son statut in-line)	X (natif conservé)	✓ (reste in-line)
Équation display courte	X	✓ (avec réglages)	✓	✓ (sans réglage)
Équation display longue	X	✓	✓	✓
Équation display 2 lignes	X	△ (rows: explicite)	△ (même limite)	✓ (rien à savoir à l’avance)
Terme isolé dans une équation	X	✓	✓ (emballé à part)	✓
Mélange texte + équation, un seul appel	△	X (tout devient un bloc)	✓ (variante récursive)	✓ (du premier coup)
Distingue add/del sur les maths	X (aucun des deux ne l’est)	possible mais jamais testé	possible mais jamais testé	✓ (E.2–E.4, teinte différente)

Méthode D = la seule sans aucun compromis relevé sur toute la matrice testée. Elle ne « corrige » pas le strike — elle change de stratégie : au lieu de décorer (ce que Typst ne sait faire que sur du texte), elle teinte (ce que Typst sait faire sur n’importe quel glyphe). C’est aussi la seule méthode qui résout, en même temps, le problème initialement posé (rendre visible un changement dans une équation) et un second problème découvert en cours de route (`add/del` indiscernables sur les maths) — avec une seule ligne de code (`text(fill: ...)`), sans mesure, boîte, ligne, ni dispatch par type de contenu.

Deux pistes tirées de l’issue GitHub `typst/typst#2200` méritent d’être retenues indépendamment de la méthode D : `math.cancel` (G.1–G.4) barre de vrais glyphes mathématiques, contrairement à `strike()` — une option pour qui préfère une vraie ligne visible sur les maths plutôt qu’une teinte (à condition d’accepter qu’elle doive être injectée équation par équation, jamais en un seul appel sur tout un passage mixte) ; et la « show rule scopée » (G.8) est une meilleure implémentation de la méthode C, sans dispatch écrit à la main. Aucune des deux ne remplace D : elles restent des façons alternatives ou complémentaires de **décorer**, quand D change la question en **teintant** à la place.

Pour les **figures** (rect, image, graphique), `text(fill: ...)` ne suffit plus (leur couleur n’est pas héritée du texte ambiant) : il faut revenir à une superposition (méthode F, section « figures supprimées »), **mais seulement là** — une figure est déjà un bloc de taille fixe, donc la mesurer et la superposer ne casse jamais rien, contrairement au texte de prose. Recolorer le contenu lui-même (F.1–F.2) a été testé et écarté : ça oblige à connaître d’avance chaque type de contenu possible, et échoue silencieusement sur tout ce qui n’est pas explicitement prévu (confirmé sur `lilaq`, F.2).

Cas (figures)	Recolorer le contenu	Superposer (voile/croix)
rect simple	✓ (si prévu explicitement)	✓
Graphique <code>lilaq</code>	X (silencieux, rien ne change)	✓
Tableau de données	non testé (aurait fallu gérer <code>table</code> aussi)	✓ (+ <code>strike()</code> natif sur les cellules, gratuit)
Légende	n’importe — c’est du texte, déjà réglé par <code>strike()</code>	idem

11 Questions ouvertes à trancher avant tout code

Les méthodes B et C (barrer/souligner **en plus**) restent dans ce document pour la comparaison, mais D change la question posée : on ne cherche plus à faire **apparaître une ligne** sur les maths, on cherche une **teinte** qui distingue add/del partout. Ce qui reste réellement à trancher avec la méthode D :

- Garde-t-on strike/underline pour le texte, en plus de la teinte ?** Tout indique que oui (E.4) : ça ne coûte rien, ça fonctionne déjà parfaitement sur la prose (méthode A), et ça donne un signal **en plus** de la couleur pour les lecteurs qui impriment en noir et blanc ou ont une deutéranomalie (où deux teintes rouges désaturées différemment peuvent être difficiles à distinguer sans la barre). Seule la couleur change de logique, pas la décoration.
 - Quelle formule exacte pour del-color() ?** `desaturate(60%).darken(15%)` est un premier essai (E.2–E.4) qui fonctionne visuellement, mais n’a pas été comparé de façon systématique à d’autres formules pour toute la palette de couleurs par relecteur (`style.typ::reviewer-color`) — certaines teintes de base pourraient mal réagir à la désaturation (le jaune assourdi devient vite illisible, par exemple, à vérifier).
 - Faut-il garder une trace de la couleur du relecteur sur une suppression, ou un gris neutre unique suffit-il ?** E.3 montre que garder l’identité (rouge assourdi reste “dans les rouges”) est possible sans perdre la distinction add/del — mais un gris fixe unique (plus simple, une seule constante) reste une option valable si l’identité du relecteur sur une suppression n’est pas jugée utile.
 - Équations multi-lignes** : n’est plus un problème avec la méthode D (D.7) — plus besoin de la question posée dans une version précédente de ce document à ce sujet.
 - Où appliquer le changement dans le code** : `mark-visual (src/marks.typ)` n’aurait plus besoin de dispatcher par type de contenu (`kind == "add"` suffit toujours) — seul le calcul de la couleur passée à `text(fill: ...)` changerait pour `kind == "del"`. Change nettement moins de choses dans le code existant que les méthodes B/C ne l’auraient demandé.
 - Figures supprimées : voile, croix, ou les deux ?** F.3–F.5 montrent les trois rendus côte à côte — c’est purement une question de goût, aucune des trois n’est plus robuste que les autres.
 - mark-figure-body doit-elle se déclencher automatiquement dans del() dès qu’un figure(...) est détecté ?** C’est un dispatch, comme la méthode C rejetée pour les équations — mais ici la détection (`body.func() == figure`) est fiable et il n’y a pas de cas mixte à gérer récursivement (une figure ne contient jamais “un peu de figure et un peu d’autre chose” à côté) : le risque relevé en C.3/C.4 ne se pose pas de la même façon ici.
 - suppress()/suppressed() restent-elles préférables pour les figures supprimées dans un template à numérotation maison (charged-ieee, §6quinquies/§6sexies) ?** La superposition (F) ne règle pas ce problème-là : la figure est toujours un vrai `figure(...)` émis, donc toujours susceptible de consommer un numéro que le template recalcule lui-même. `suppress()` reste la bonne réponse quand ce risque existe ; la méthode F vise plutôt le cas où l’auteur veut **montrer** la figure supprimée (barrée/voilée), pas seulement la mentionner par une note.
 - Teinte seule (D) ou vraie ligne native sur les maths (math.cancel, G.1–G.4) ?** Les deux résolvent la visibilité d’un changement dans une équation, mais différemment : D ne touche à rien d’autre que la couleur (marche du premier coup sur un passage mixte, case D.9, aucune limite de largeur ou de multi-rangées) ; `cancel()` donne une vraie barre sur les glyphes eux-mêmes, plus proche visuellement de ce qu’on attend d’un « texte barré », mais avec deux limites propres à la fonction elle-même, confirmées y compris en l’injectant par dispatch manuel plutôt que par show rule (cases H.1–H.5) : elle ne gère pas les équations multi-rangées (une seule ligne traverse les deux rangées, H.3) et elle rend une équation inline longue de nouveau insécable, avec le même risque de débordement en colonne étroite que la superposition (H.4). Les deux ne s’excluent pas complètement : `del-color()` (teinte) pourrait s’appliquer à tout le passage et `cancel()` être ajouté en plus, à la main, sur une équation courte ponctuelle où l’auteur veut une vraie barre — mais pas comme mécanisme automatique et général dans `del()`, vu ces deux limites.